University of Peradeniya

CO326 Edge AI + Industrial IoT Mini Project

Edge AI Based Predictive Maintenance System for an HVAC Blower Fan

Project Area	Edge AI + Industrial IoT
Project Title	HVAC Blower Fan Predictive Maintenance System
Group Number	Group 16
Group Members	E/20/262 - Nanayakara A.T.L E/20/288 – Perera M.C.S.J E/20/449 – Wijewardhana A.R.S.S E/20/279 – Panawennage L.S.

Table of Contents

1. Introduction

Industrial fans, motors, and rotating machines often fail due to mechanical imbalance, abnormal vibration, overload conditions, or long-term wear. Traditional maintenance approaches usually depend on scheduled inspections or reactive repair after failure. In contrast, predictive maintenance uses sensor data and intelligent analysis to detect early signs of faults before serious damage occurs. This project applies that idea to an HVAC blower fan by using an ESP32 as an edge computing device.

The system measures fan vibration using an MPU6050 sensor and current draw using an ACS712 current sensor. These readings are sampled at approximately 100 Hz and converted into feature windows. A trained K-Means anomaly detection model is exported to the ESP32 and used during live operation. The ESP32 classifies the fan condition as normal, warning, or critical and publishes the results as JSON messages over MQTT.

On the IIoT side, MQTT messages are received by a Docker-based stack. Node-RED processes the data, InfluxDB stores the time-series values, and Grafana visualizes system behavior. The dashboard provides gauges, charts, and health status panels that can be used during the final live demonstration.

2. Objectives and Scope

The main objective is to design and implement a complete Edge AI based Industrial IoT system that can acquire sensor data, process it locally, apply anomaly detection, communicate using MQTT, and display live results through a dashboard.

- Acquire real fan vibration and current data using ESP32, MPU6050, and ACS712 sensors.
- Train an anomaly detection model using labelled normal, warning, and critical operating states.
- Export the trained model parameters to the ESP32 and run inference at the edge.
- Publish sensor values, features, anomaly score, and health status using MQTT messages.
- Use Node-RED and Grafana dashboards to visualize real-time fan health and alerts.
- Maintain the project using a clean GitHub repository structure and meaningful documentation.

3. System Architecture

The overall architecture follows the required flow: Sensor/Data -> Edge AI Processing -> MQTT -> Node-RED -> Dashboard. The system is separated into two major stages. Stage 1 focuses on data collection, feature engineering, model training, validation, and export of trained parameters. Stage 2 focuses on real-time inference, MQTT communication, data storage, dashboard visualization, alerting, and relay control.

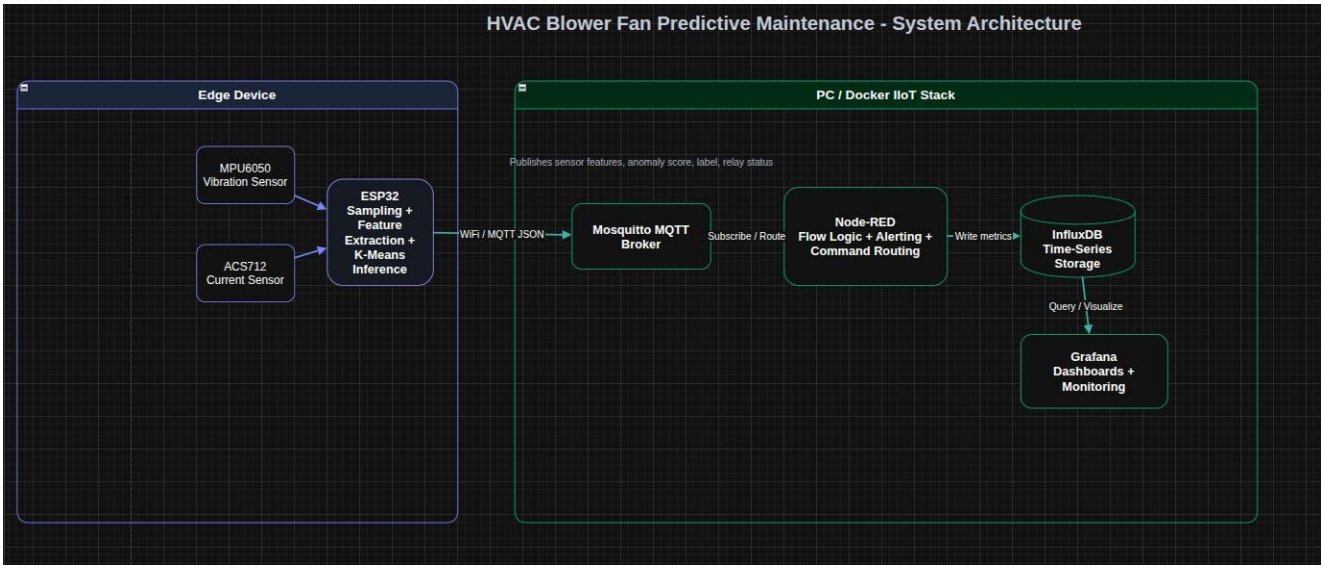


Figure 1: System Architecture

3.1 Main Components

Layer	Technology / Component	Function
Edge device	ESP32 WROOM-32	Reads sensors, extracts features, runs K-Means inference, publishes MQTT messages.
Sensors	MPU6050 and ACS712	Measures vibration and fan current draw.
AI model	K-Means anomaly detection	Calculates anomaly score and labels fan state.
Communication	MQTT / Mosquitto	Lightweight publish-subscribe messaging between edge and dashboard stack.
Flow processing	Node-RED	Parses JSON payloads, routes data, creates alerts, and controls relay endpoints.
Storage	InfluxDB 2.7	Stores time-series sensor and anomaly values.
Visualization	Grafana	Displays live gauges, charts, health status, and history.
Actuation	Relay driver circuit	Optional fan shutdown during critical fault detection.

MPU6050 Sensor

ACS712 Sensor

4. Hardware and Sensor Setup

The hardware prototype uses an ESP32 DevKit v1 as the main edge device. The MPU6050 vibration sensor is connected using I2C with GPIO 21 as SDA and GPIO 22 as SCL. The ACS712 current sensor is connected to GPIO 34, which is an ADC input pin. The HVAC blower fan is powered using a separate 12 V supply, while the sensor circuit is connected to the ESP32.

Component	Connection / Pin	Purpose
MPU6050 VCC	ESP32 3V3	Power for vibration sensor.
MPU6050 GND	ESP32 GND	Common ground.
MPU6050 SDA	ESP32 GPIO 21	I2C data line.
MPU6050 SCL	ESP32 GPIO 22	I2C clock line.
ACS712 VCC	ESP32 VIN / 5 V	Power for current sensor module.
ACS712 GND	ESP32 GND	Common ground.
ACS712 OUT	ESP32 GPIO 34	Analog current reading.
Relay control	ESP32 GPIO 26 through transistor driver	Optional fan ON/OFF control.

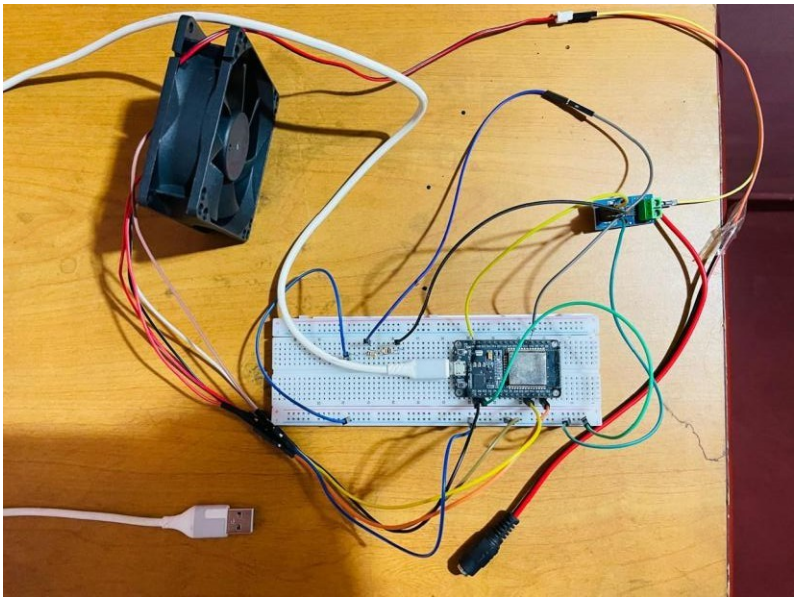


Figure 2: Hardware wiring setup for sensor acquisition and relay control.

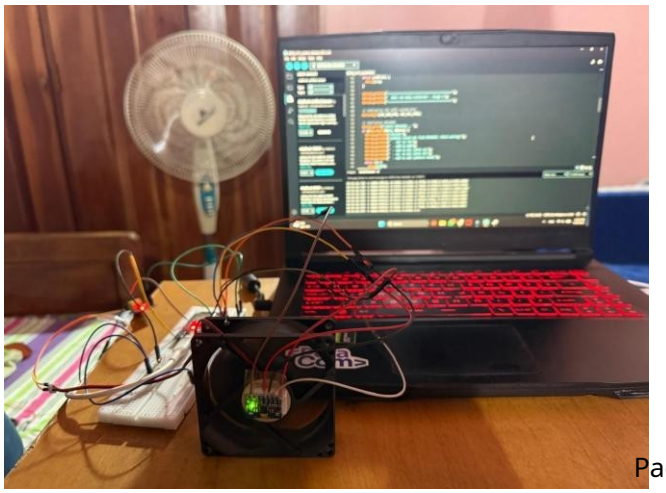
4.1 Safety Considerations

Because the project involves a rotating fan and fault injection using additional blade weight, safety was considered during data collection and testing. The fan should be switched off before attaching any weight, the weight must be secured firmly, and the fan should be stopped immediately if there is excessive movement or if the attached object appears unstable. The relay driver also includes flyback protection using a diode to protect the transistor and ESP32 from voltage spikes generated by the relay coil.

5. Data Acquisition and Feature Engineering

During Stage 1, labelled sensor data was collected under different fan conditions. The ESP32 data collector firmware reads sensor values at approximately 100 Hz and prints data over Serial. A Python script records this output into CSV files for each condition. The dataset includes normal operation, normal-low operation with restricted airflow, warning-level imbalance, and critical-level imbalance.

Fan State	Description	Expected Duration / Purpose
Normal	Clean fan running freely.	Baseline healthy fan behavior.
Normal Low	Clean fan with restricted airflow.	Healthy fan under different load condition.
Warning	Mild imbalance using approximately 8-12 g attached weight.	Early fault or warning condition.
Critical	Severe imbalance using approximately 20-30 g attached weight.	High-risk condition requiring alert or shutdown.



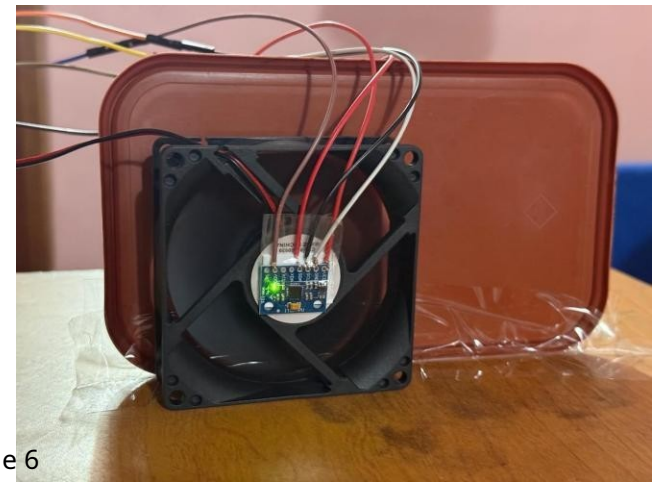


Figure 3: Sensor data acquisition through ESP32 and serial logging.

5.1 Feature Windowing

Raw sensor samples are converted into feature vectors using a sliding window method. A window of 256 samples represents approximately 2.56 seconds of fan behavior at 100 Hz. A 50% overlap between windows improves continuity and increases the number of training examples. Each feature vector contains vibration, current, and spectral characteristics.

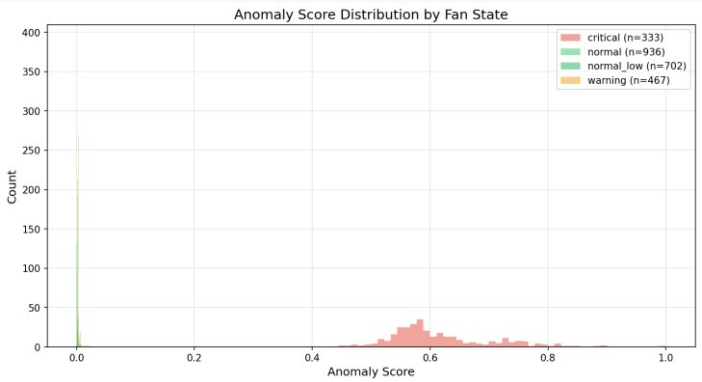
Feature Group	Examples	Reason for Use
Vibration features	vib_rms, vib_peak, vib_crest, vib_kurt	Captures vibration energy, peak behavior, and imbalance patterns.
Current features	cur_rms, cur_std	Captures load and electrical variation of the fan.
Frequency features	dom_freq, spec_rms, spec_cent, band1, band2, band3	Captures frequency-domain behavior that changes during imbalance faults.

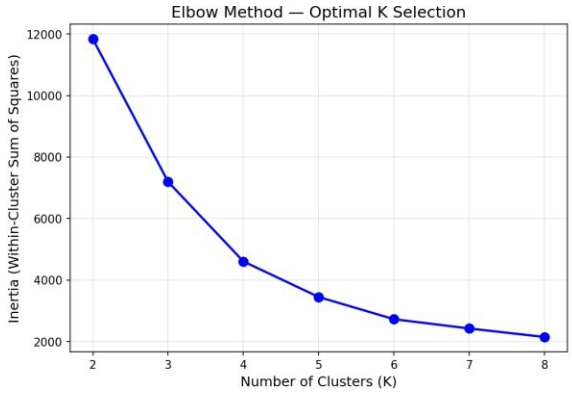
6. Edge AI / Anomaly Detection Implementation

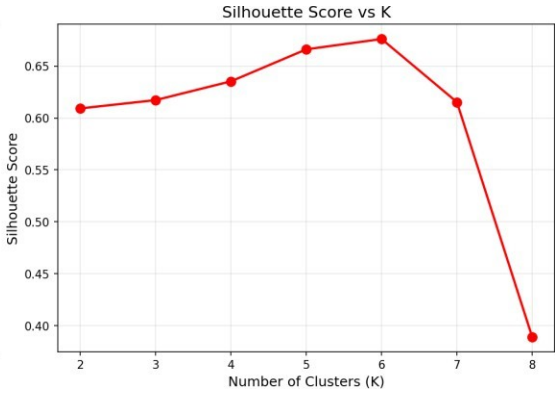
The AI component uses K-Means clustering trained mainly on healthy fan behavior. After training, the model centroids, scaler parameters, and threshold values are exported into a C++ header file and included in the ESP32 inference firmware. This allows the ESP32 to classify fan condition locally without needing continuous cloud processing.

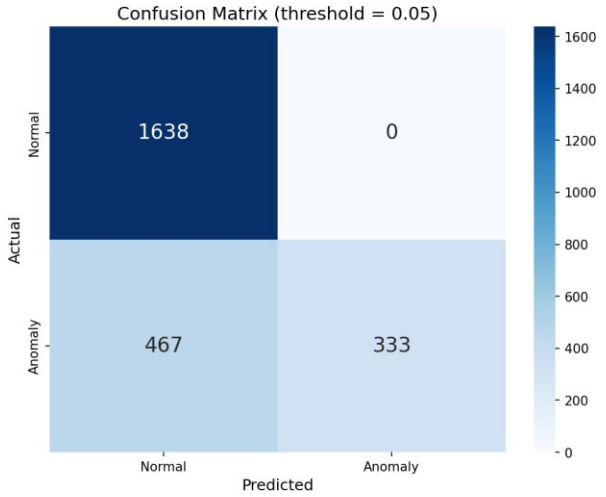
For each 2.56-second sensor window, the ESP32 extracts 12 features, applies standard scaling, calculates the distance to the nearest K-Means centroid, converts this distance into an anomaly score, and classifies the state. The anomaly result is then included in the MQTT JSON payload.

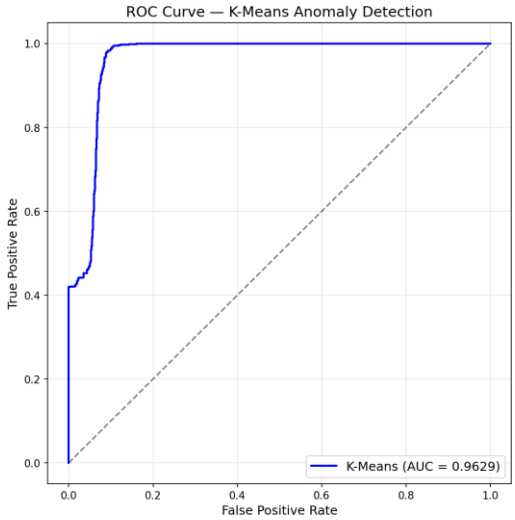
```
Sensor readings -> 256-sample window -> 12 extracted features -> StandardScaler -> K-Means nearest
centroid -> anomaly score -> NORMAL / WARNING / CRITICAL
```

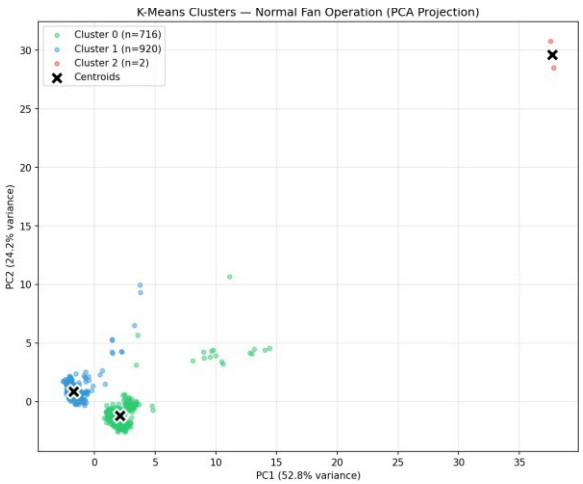


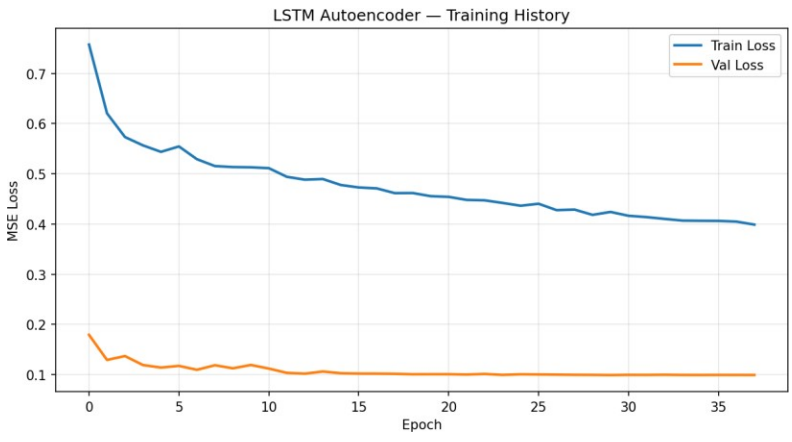

Elbow Method — Optimal K Selection
Inertia (Within-Cluster Sum of Squares)
Number of Clusters (K)

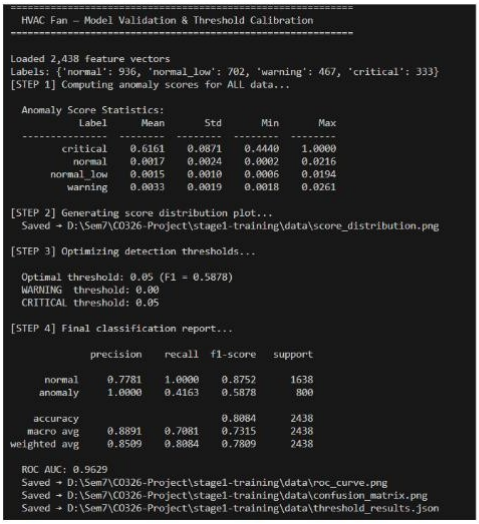

Silhouette Score vs K
Silhouette Score
Number of Clusters (K)


Confusion Matrix (threshold = 0.05)
1638
0
467
333
Normal
Anomaly
Actual
Predicted


ROC Curve — K-Means Anomaly Detection
True Positive Rate
False Positive Rate
K-Means (AUC = 0.9629)


K-Means Clusters — Normal Fan Operation (PCA Projection)
Cluster 0 (n=716)
Cluster 1 (n=920)
Cluster 2 (n=2)
Centroids
PC1 (52.8% variance)
PC2 (24.2% variance)


LSTM Autoencoder — Training History
Train Loss
Val Loss
MSE Loss
Epoch



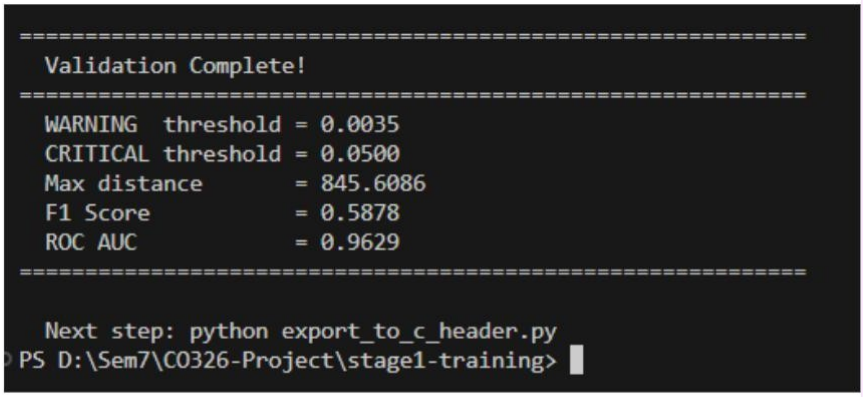


Figure 4: Model validation plots showing anomaly detection performance.

6.1 Classification Logic

Condition	Score Range / Logic	System Response
Normal	Anomaly score below warning threshold.	Dashboard displays normal status; fan continues running.
Warning	Anomaly score exceeds warning threshold but is below critical threshold.	Warning alert is generated and dashboard status changes.
Critical	Anomaly score exceeds critical threshold.	Critical alert is generated; relay can automatically stop the fan if connected.

7. MQTT Communication and IIoT Pipeline

MQTT is used as the main communication protocol because it is lightweight, supports publish-subscribe communication, and is suitable for IoT systems. The ESP32 publishes JSON messages containing sensor features, anomaly score, anomaly label, relay status, and signal strength. The broker routes messages to Node-RED, where the data is parsed, displayed, stored, and used for alert logic.

7.1 MQTT Topics Used

Topic	Direction	Payload / Purpose
hvac/fan01/data	ESP32 -> MQTT Broker	Live sensor features, anomaly score, anomaly label, relay state, and RSSI.
hvac/fan01/cmd/relay	Dashboard/Node-RED -> ESP32	Relay ON/OFF control command.
alerts/groupXX/hvac-fan/status	Node-RED -> Dashboard / debug	Suggested final CO326 alert topic for warning and critical health messages.
sensors/groupXX/hvac-fan/data	ESP32 -> MQTT Broker	Suggested final CO326-compliant sensor topic format for submission.

7.2 Docker-Based IIoT Stack

The Stage 2 live system runs several services using Docker Compose. This makes the system easier to start, reproduce, demonstrate, and maintain. The stack includes Mosquitto for MQTT, Node-RED for flow processing, InfluxDB for time-series storage, and Grafana for visualization.

Service	Port	Role

Mosquitto	1883	MQTT broker for routing edge device messages.
Node-RED	1880	Flow processing, alert logic, dashboard endpoints, and data routing.
InfluxDB	8086	Time-series database for sensor and anomaly data.
Grafana	3000	Live monitoring dashboard with charts and gauges.

```
PS D:\Sem7\CO326-Project\stage2-running> docker compose up -d
[+] Running 44/44
 ✓ grafana Pulled                                                125.1s
 ✓ nodered Pulled                                                182.0s
 ✓ mosquitto Pulled                                               34.7s
 ✓ influxdb Pulled                                               153.7s
[+] Running 7/7
 ✓ Network stage2-running_hvac_net       Created                   0.1s
 ✓ Volume stage2-running_influxdb_data   Created                   0.0s
 ✓ Volume stage2-running_grafana_data    Created                   0.0s
 ✓ Container influxdb                    Started                   3.8s
 ✓ Container mosquitto                   Started                   3.8s
 ✓ Container grafana                     Started                   1.7s
 ✓ Container nodered                     Started                   1.7s
PS D:\Sem7\CO326-Project\stage2-running>
```

Figure 5: Docker Compose services used for the IIoT dashboard stack.

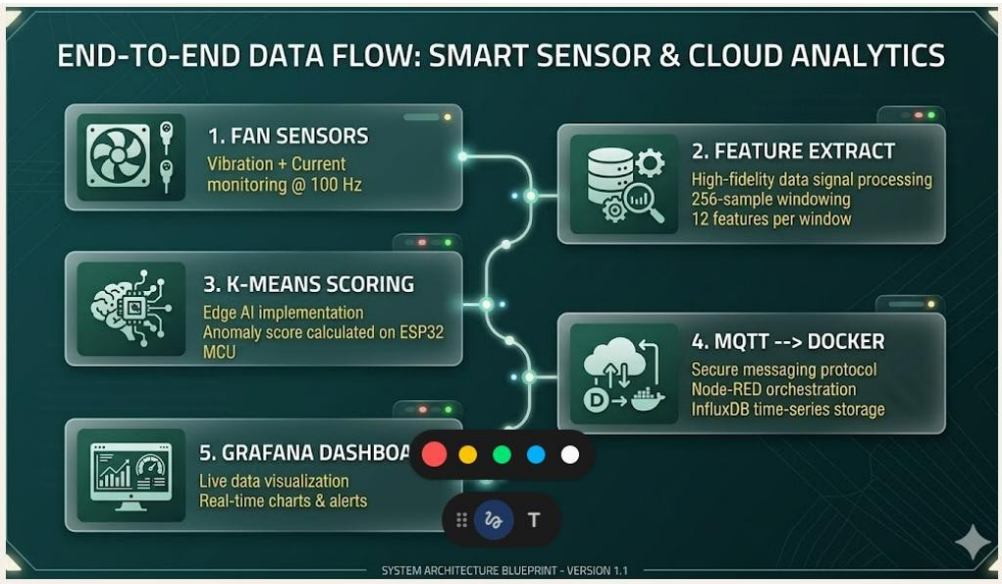


Figure 6: End-To-End data flow

8. Dashboard Visualization

The dashboard layer is designed to clearly communicate the current health condition of the fan. Grafana displays real-time gauges and time-series panels, while Node-RED can be used for flow-level monitoring and alert visualization. The dashboard updates frequently, making it suitable for the final live demonstration.

Dashboard Panel	Displayed Information
Anomaly Score Gauge	Current anomaly score with green, warning, and critical zones.
Vibration RMS Gauge	Current vibration magnitude of the fan.
Fan Health Status	Text status such as NORMAL, WARNING, or CRITICAL.

Relay / Fan Power Status	Shows whether the fan relay is ON or OFF.
Anomaly Time-Series Chart	Historical anomaly score changes and threshold behavior.
Vibration and Current Charts	Live fan vibration and current behavior over time.
Spectral Band Power	Frequency-band behavior used to understand imbalance patterns.

Figure 6: Real-time fan health monitoring dashboard.

9. Testing and Results

Testing was carried out to verify the complete system flow from sensor acquisition to dashboard visualization. The main test areas were hardware reading, feature extraction, model inference, MQTT publishing, Docker service status, Node-RED data flow, InfluxDB storage, Grafana visualization, alert status changes, and relay response.

9.1 Test Plan

Test Case	Expected Result	Evidence to Add
Sensor reading test	MPU6050 and ACS712 values change when fan runs or sensor is moved.	Serial Monitor screenshot.
MQTT publish test	JSON messages arrive every approximately 2.56 seconds.	Node-RED debug or mosquitto_sub screenshot.
Docker stack test	All containers show Up status.	docker compose ps screenshot.
Dashboard test	Gauges and charts update with live readings.	Grafana screenshot.
Warning fault test	Mild imbalance increases anomaly score and changes health status.	Dashboard before/after screenshot.
Critical fault test	Severe imbalance triggers critical status and optional relay shutdown.	Fault injection and relay status screenshot.

9.2 Result Summary

The project successfully demonstrates a live predictive maintenance workflow. During normal operation, the anomaly score remains low and the dashboard shows a healthy state. When additional weight is attached to the fan blade, vibration features change and the K-Means distance increases. This causes the anomaly score to move into warning or critical ranges. In the critical state, the system can trigger alerts and stop the fan through relay control if the relay circuit is connected.

Operating State	Observed System Behavior	Interpretation
Normal	Low anomaly score and stable vibration/current readings.	Healthy operation.
Warning imbalance	Increased anomaly score and warning status.	Early fault detected.
Critical imbalance	High anomaly score, critical status, and possible relay shutdown.	Unsafe fan condition detected.

Recovery	Score decreases after removing added weight.	System returns to normal monitoring state.

10. Challenges and Solutions

Challenge	Solution / Mitigation
Sensor noise and inconsistent readings	Used window-based feature extraction instead of relying on raw single samples.
ESP32 ADC and sensor voltage compatibility	Used safe wiring practices and voltage divider where required for ACS712 output.
Model deployment on microcontroller	Exported K-Means centroids and scaler parameters to a C++ header file.
MQTT connectivity issues	Verified WiFi, broker IP, port 1883 access, and same-network connectivity.
Dashboard no-data issues	Checked Node-RED flow, MQTT subscription, InfluxDB token, bucket, and Grafana data source settings.
Safe fault injection	Used controlled weights and stopped the fan before attaching or removing them.

11. Future Improvements

- Deploy the system with the instructor-provided central MQTT broker and final CO326 topic naming convention.
- Add more fan fault categories such as bearing wear, obstruction, motor overload, and loose mounting.
- Improve the AI model using more training data and compare K-Means with Isolation Forest or TinyML autoencoder models.
- Add a mobile notification or email alert for critical fault detection.
- Use secure MQTT with authentication and TLS for a production-level IIoT deployment.
- Improve the enclosure and wiring to make the prototype safer and more suitable for industrial demonstration.

12. Member Contribution

Member Name	Reg. No.	Project Contribution	Report Section Created / Supported
Wijewardhana A.R.S.S	E/20/449	Worked on ESP32 sensor setup, MPU6050 vibration sensor connection, ACS712 current sensor wiring, and real-time data acquisition.	Hardware Setup, Data Acquisition, System Architecture
Nanayakkara A.T.L	E/20/262	Developed the Python-based ML workflow, feature extraction, K-Means anomaly detection model training, validation, and model export for ESP32.	Edge AI Implementation, Model Training, Results and Evaluation
Perera M.C.S.J	E/20/288	Configured MQTT communication, Docker services, Mosquitto broker, Node-RED flow, alert logic, and data pipeline.	MQTT Communication, Docker Environment, Alert Integration
Lashan Panawenna	E/20/282	Designed dashboard visualization using Node-RED/Grafana, tested live data flow, captured screenshots, and supported final documentation formatting.	Dashboard Visualization, Testing, Screenshots, Challenges and Future Improvements

13. Conclusion

This project successfully implements a complete Edge AI and Industrial IoT predictive maintenance system

for an HVAC blower fan. It satisfies the CO326 mini project requirements by acquiring real sensor data, processing the data

at the edge, applying AI-based anomaly detection, communicating using MQTT, and visualizing results through a dashboard. The use of Docker, Node-RED, InfluxDB, Grafana, GitHub, and ESP32-based firmware makes the project practical, reproducible, and suitable for a live demonstration.

The most important outcome is that the system can detect abnormal fan behavior locally and communicate meaningful status information to the dashboard. This demonstrates how edge intelligence can reduce dependency on cloud processing, improve real-time response, and support predictive maintenance in industrial environments.